

THINGS TO KNOW:

1. Lab report must contain following sections: (order must be maintained)
 - a) Title /Question
 - b) Theory: The brief overview of the concept /techniques/syntax/idea/tools etc used
 - c) Steps and Commands: The steps performed along with involved commands
 - d) Output: Screenshot of the output
2. Output screen should be captured (use snipping tool), printed and attached in the report.
3. The output screen should reflect the unique identity of each students for that include the following things in commands (to print message in screen) and/or names and/ or paths :
 - Personal Information (Name, Roll No. and Section)
4. Contents should be written on single side of A4 sized paper.
5. Cover page and contents page should be attached in the report appropriately.
6. The works must be submitted within specified deadline.

Content Page Format

Subject: Database Administration

List of Lab Works

Lab No.	Title /Question	Submission Date	Signature
1(a)	This is sample title	2079/03/15	
1(b)	This is another title	2079/03/17	

Lab works / Part-5 / DBA/ BIT-6th semester

First Create PL/SQL stored procedure that displays the information about lab no. and your personal information in required places (CDB and PDB) and execute the stored procedure at beginning or end of the each lab task (for those lab works where SQL Plus is used)

Sample PL/SQL stored procedure:

```
CREATE OR REPLACE PROCEDURE my_information IS
BEGIN
    DBMS_OUTPUT.PUT_LINE('Name: Ram Prasad Sharma/Roll No.: 102 / Section: A');
END my_information;
/
```

Execution:

```
SET SERVEROUTPUT ON;
EXEC my_information;
```

17. Database Maintenance and Diagnostic Monitoring in Oracle 19c

- a) **Preparation and Configuration:** Connect as SYS and check statistics_level via SHOW PARAMETER statistics_level;. Ensure it is set to TYPICAL or ALL. If set to BASIC, execute ALTER SYSTEM SET statistics_level=TYPICAL SCOPE=BOTH;. Provide SQL commands and screenshots before and after the change.
- b) **View the Alert Log:** Locate the Alert Log path using SELECT value FROM v\$diag_info WHERE name='Diag Trace'; or launch an adrci terminal session (show alert). Trigger an intentional diagnostic entry (e.g., attempt creating a tablespace referencing an invalid file path). Provide commands, ADRCI output, and a screenshot of the log snippet.
- c) **Automatic Workload Repository and ADDM:** Create two manual snapshots using EXEC DBMS_WORKLOAD_REPOSITORY.CREATE_SNAPSHOT(); around a workload interval. Execute @\$ORACLE_HOME/rdbms/admin/addmrpt.sql (or DBMS_ADVISOR) between the generated SNAP_ID pair. Provide execution commands, snapshot listings, and screenshots of findings and recommendations.
- d) **Monitor an Oracle Database:** Query dynamic performance views (V\$SESSION, V\$SESSTAT, V\$SYSTEM_EVENT) to extract real-time session activity and top system wait events. List the top 3 wait events impacting the instance and explain their root causes. Include SQL queries and output screenshots.
- e) **Use the Advisors:** Run the SQL Tuning Advisor against an unindexed query using DBMS_SQLTUNE.CREATE_TUNING_TASK and EXECUTE_TUNING_TASK.

Run the Segment Advisor on HR.EMPLOYEES via DBMS_ADVISOR to identify reclaimable space. Provide scripts and report output screenshots.

- f) **Set Up Notification Rules:** Set up a threshold-based alert for tablespace usage exceeding 85% using DBMS_SERVER_ALERT.SET_THRESHOLD or EM Express. Query DBA_THRESHOLDS and DBA_OUTSTANDING_ALERTS to verify trigger behavior. Include SQL commands and verification screenshots.

18. Performance Management and Memory Tuning in Oracle 19c

- a) **Tuning Information Sources:** Connect as SYS and inspect key tuning views: V\$SYSSTAT, V\$SESSION_EVENT, V\$SQL, and V\$SGA_TARGET_ADVICE. Display current memory settings (sga_target, pga_aggregate_target, memory_target) using SHOW PARAMETER. Provide SQL commands and parameter screenshots.
- b) **Performance Monitoring and Tuning Activities:** Connect as HR and execute a loop containing mixed DML/query statements on EMPLOYEES and DEPARTMENTS. Simultaneously monitor execution via V\$SESSION_LONGOPS and V\$SQLAREA to spot queries with high BUFFER_GETS and EXECUTIONS. Include the workload script and monitoring screenshots.
- c) **Performance Planning and Instance Tuning:** Query V\$BUFFER_POOL_STATISTICS and V\$LIBRARYCACHE to evaluate the Buffer Cache Hit Ratio and Library Cache Hit Ratio. Review sizing advice using V\$SGA_TARGET_ADVICE and V\$PGA_TARGET_ADVICE. Document sizing recommendations alongside hit-ratio SQL outputs.
- d) **Managing Memory:** Verify if Automatic Shared Memory Management (ASMM) or Automatic Memory Management (AMM) is active. Temporarily set fixed values for db_cache_size and shared_pool_size via ALTER SYSTEM, observe changes in V\$SGA_DYNAMIC_COMPONENTS, and revert to automatic management. Include commands, before/after screenshots, and a summary of manual vs. automatic trade-offs.
- e) **Manage Private Temporary Tables (PTT):** Connect as HR and create a Private Temporary Table:

```
CREATE PRIVATE TEMPORARY TABLE ora$ppt_test (  
    id NUMBER,  
    val VARCHAR2(50)  
) ON COMMIT DROP DEFINITION;
```

Insert rows, query within session, issue a COMMIT, and verify the definition is removed. Repeat using ON COMMIT PRESERVE DEFINITION to compare lifecycle behaviors. Include commands and execution screenshots.

- f) **Validation and Cleanup:** Revert memory settings to automatic defaults, drop test objects, and query V\$PGASTAT for final memory verification. Include cleanup commands and verification screenshots.

19. SQL Tuning and the Oracle Optimizer

- a) **Preparation and Configuration:** Connect as SYS and grant privileges: GRANT ADVISOR TO HR;. Check SHOW PARAMETER optimizer_mode (default ALL_ROWS). Provide SQL commands and parameter screenshots.
- b) **Identify a Poorly Performing SQL Statement:** Connect as HR and execute a query filtering on an unindexed function (e.g., WHERE UPPER(last_name) = 'KING'). Generate the execution plan using EXPLAIN PLAN FOR ... and display it with SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);. Note the full table scan cost. Include query scripts and plan screenshots.
- c) **Cancel a Running SQL Statement:** In Session 1 (HR), initiate a long-running query or PL/SQL delay loop (DBMS_SESSION.SLEEP). In Session 2 (SYSDBA), locate the statement's SID and SERIAL# via V\$SESSION and execute:

```
ALTER SYSTEM CANCEL SQL 'sid,serial';
```

Verify cancellation without session disconnection. Include lookup queries, cancellation syntax, and ORA-01013 error screenshots.

- d) **SQL Tuning Advisor and Recommendations:** Run DBMS_SQLTUNE on the query from part (b). Display the report using REPORT_TUNING_TASK. Apply the recommended fix (create index or accept SQL Profile using DBMS_SQLTUNE.ACCEPT_SQL_PROFILE). Show execution plan improvements with DBMS_XPLAN. Include advisor scripts and comparison screenshots.
- e) **SQL Plan Directives and Adaptive Execution Plans:** Query DBA_SQL_PLAN_DIRECTIVES to inspect optimizer directives. Re-run the query and inspect execution details using DBMS_XPLAN.DISPLAY_CURSOR(format => 'ALLSTATS LAST') to check for adaptive plan notes. Explain how directives and adaptive plans stabilize execution. Include SQL commands and output screenshots.
- f) **Validation and Cleanup:** Summarize cost/cardinality changes in a comparison table. Remove temporary indexes or SQL profiles (DBMS_SQLTUNE.DROP_SQL_PROFILE). Include screenshots of cleanup operations.

20. Moving Data with Data Pump and SQL*Loader

- a) **Preparation: Directory Objects:** Connect as SYS. Create a directory object:

```
CREATE DIRECTORY dpump_dir AS '/u01/app/oracle/dpdump';
```

Grant access: GRANT READ, WRITE ON DIRECTORY dpump_dir TO HR;. Verify via DBA_DIRECTORIES. Provide SQL commands and setup screenshots.

- b) **Data Pump Export:** Run expdp from the OS terminal to export the HR schema tables, generating a log file. Review the export log for completion status. Include terminal execution commands and log screenshots.
- c) **Data Pump Views and Interactive Mode:** Start a background export job and query DBA_DATAPUMP_JOBS and DBA_DATAPUMP_SESSIONS. Attach to the running job using expdp attach=job_name, issue STATUS, and test interactive commands (STOP_JOB / CONTINUE_CLIENT). Include dictionary queries and interactive mode screenshots.
- d) **Data Pump Import:** Rename target tables in HR (RENAME EMPLOYEES TO EMPLOYEES_BAK;). Run impdp to restore tables using TABLE_EXISTS_ACTION=REPLACE. Compare row counts against EMPLOYEES_BAK. Include import commands and verification query screenshots.
- e) **SQL*Loader: Control File and Data File:** Create a flat file (new_employees.csv) containing sample data. Construct a SQL*Loader control file defining table mappings and field delimiters. Execute sqlldr from the OS prompt. Include control file syntax, command invocation, and load summary screenshots.
- f) **Record Filtering and External Tables:** Add a WHEN (department_id = '50') clause to the control file and execute a filtered load. Next, construct an external table using ORGANIZATION EXTERNAL over the CSV file and query it via SQL. Include modified control files, load outputs, and external table query screenshots.

21. Database Resource Manager (DBRM)

- a) **Preparation and Configuration:** Connect as SYS. Check active resource plans using SHOW PARAMETER resource_manager_plan and V\$RSRC_PLAN. Inspect DEFAULT_MAINTENANCE_PLAN via DBA_RSRC_PLANS. Include commands and verification screenshots.
- b) **Create Consumer Groups and Resource Plan:** Execute a PL/SQL block using DBMS_RESOURCE_MANAGER to:
 - Create a Pending Area (CREATE_PENDING_AREA).
 - Create consumer groups (HR_GROUP, BATCH_GROUP).
 - Create a resource plan (LAB_PLAN).
 - Submit the Pending Area (SUBMIT_PENDING_AREA).

Verify using DBA_RSRC_CONSUMER_GROUPS and DBA_RSRC_PLANS. Include PL/SQL blocks and dictionary screenshots.

- c) **Create Resource Plan Directives:** Create a new pending area and define directives allocating CPU (e.g., 70% to HR_GROUP, 30% to BATCH_GROUP) and setting

PARALLEL_DEGREE_LIMIT_P1. Validate and submit. Verify with DBA_RSRC_PLAN_DIRECTIVES. Include setup scripts and confirmation screenshots.

- d) **Assign Users and Activate Plan:** Map user HR to HR_GROUP using DBMS_RESOURCE_MANAGER.SET_CONSUMER_GROUP_MAPPING. Activate the plan using:

```
ALTER SYSTEM SET RESOURCE_MANAGER_PLAN = 'LAB_PLAN';
```

Verify execution in V\$RSRC_PLAN. Include assignment scripts and parameter screenshots.

- e) **Test and Observe Resource Allocation:** Execute a CPU-intensive query under the HR session. Monitor resource usage in V\$RSRC_CONSUMER_GROUP and check the RESOURCE_CONSUMER_GROUP column in V\$SESSION. Evaluate whether CPU allocation aligns with defined directives. Include monitoring queries and output screenshots.
- f) **Validation and Cleanup:** Reset RESOURCE_MANAGER_PLAN to its default value. Drop LAB_PLAN and associated consumer groups within a pending area. Include reversion syntax and cleanup screenshots.

22. Oracle Scheduler: Jobs, Job Classes, and Event-Based Schedules

- a) **Preparation and Access Rights:** Connect as SYS. Grant permissions: GRANT CREATE JOB TO HR;. Verify in DBA_SYS_PRIVS. Query DBA_SCHEDULER_JOBS, DBA_SCHEDULER_PROGRAMS, and DBA_SCHEDULER_SCHEDULES for HR. Include SQL commands and privilege screenshots.
- b) **Create a Time-Based Job:** Connect as HR and create a repeating job using DBMS_SCHEDULER.CREATE_JOB:

```
BEGIN

DBMS_SCHEDULER.CREATE_JOB (

  job_name      => 'PURGE_OLD_LOGS_JOB',
  job_type      => 'PLSQL_BLOCK',
  job_action    => 'INSERT INTO log_table VALUES (SYSDATE); COMMIT;',
  start_date    => SYSTIMESTAMP,
  repeat_interval => 'FREQ=MINUTELY; INTERVAL=2',
  enabled       => TRUE

);

END;
```

/

Allow the job to run at least twice and verify execution via DBA_SCHEDULER_JOB_RUN_DETAILS. Include job setup scripts and run log screenshots.

c) Create a Job Class: Create a job class using DBMS_SCHEDULER.CREATE_JOB_CLASS with a target consumer group and logging level. Reassign PURGE_OLD_LOGS_JOB to this class. Verify using DBA_SCHEDULER_JOB_CLASSES. Include PL/SQL blocks and configuration screenshots.

d) Event-Based Schedule: Create an event schedule using DBMS_SCHEDULER.CREATE_EVENT_SCHEDULE linked to an AQ queue condition (EVENT_CONDITION). Explain how event signals trigger job execution. Include creation scripts and triggered execution screenshots.

e) Monitor and Manage Jobs: Query DBA_SCHEDULER_RUNNING_JOBS during job execution and inspect status in DBA_SCHEDULER_JOB_LOG. Practice stopping and disabling active jobs:

```
EXEC DBMS_SCHEDULER.STOP_JOB('PURGE_OLD_LOGS_JOB');
```

```
EXEC DBMS_SCHEDULER.DISABLE('PURGE_OLD_LOGS_JOB');
```

Include monitoring output and job-control screenshots.

f) Validation and Cleanup: Drop created jobs, job classes, and schedules (DROP_JOB, DROP_JOB_CLASS, DROP_SCHEDULE). Confirm removal using DBA_SCHEDULER_* views. Include cleanup commands and verification screenshots.

23. Space Management: Segments, Extents, and Row Chaining

a) Preparation and Configuration: Connect as SYS. Query DBA_TABLESPACES (SEGMENT_SPACE_MANAGEMENT column) to check if the USERS tablespace uses Auto Segment Space Management (ASSM). Include SQL commands and configuration screenshots.

b) Examine Segment Types and Allocated Extents: Connect as HR and query DBA_SEGMENTS for EMPLOYEES to retrieve SEGMENT_TYPE, BYTES, BLOCKS, and EXTENTS. Query DBA_EXTENTS to inspect assigned physical extents. Include SQL statements and dictionary screenshots.

c) Allocate Additional Space: Manually allocate space to EMPLOYEES:

```
ALTER TABLE EMPLOYEES ALLOCATE EXTENT (SIZE 1M);
```

Re-query DBA_SEGMENTS and DBA_EXTENTS to verify increased extent allocations. Include allocation syntax and before/after screenshots.

d) Row Chaining and Migration:

- Create a table with low PCTFREE and multiple VARCHAR2 columns.
- Insert initial rows with minimal column lengths.
- Execute UPDATE statements expanding column sizes to force row migration.
- Run \$ORACLE_HOME/rdbms/admin/utlchain.sql to build the CHAINED_ROWS table.
- Analyze the table:

```
ANALYZE TABLE test_chain LIST CHAINED ROWS INTO CHAINED_ROWS;
```

- Query CHAINED_ROWS to confirm migrated rows.

Include setup scripts, update queries, and analysis screenshots.

e) Create Tables Without Segments: Create a table using deferred segment creation:

```
CREATE TABLE test_deferred (id NUMBER) SEGMENT CREATION DEFERRED;
```

Verify SEGMENT_CREATED = 'NO' in DBA_TABLES. Insert a row and confirm automatic segment allocation in DBA_SEGMENTS. Include before/after SQL commands and screenshots.

f) Validation and Cleanup: Drop CHAINED_ROWS and created test tables. Document extent management recommendations. Include cleanup commands and verification screenshots.

24. Oracle Support: Support Workbench, Patches, and Security Updates

a) Preparation and Configuration: Connect as SYS. Check the Automatic Diagnostic Repository path:

```
SELECT value FROM v$diag_info WHERE name = 'ADR Home';
```

Launch adrci and run show homes to inspect directory structures (incident, trace, alert). Include diagnostic queries and ADRCI terminal screenshots.

b) Support Workbench (Incident Investigation): List ADR incidents using adrci> show incident. Package an incident for Oracle Support using Incident Packaging Services (IPS):

```
adrci> IPS CREATE PACKAGE INCIDENT <incident_id>;
```

```
adrci> IPS GENERATE PACKAGE <package_id> IN /tmp;
```

Include ADRCI listing outputs and package generation screenshots.

c) Register for Security Updates: Document the process for configuring Oracle Security Alert notifications in EM Express (**Configuration > My Oracle Support > Register**).

- d) Work with Oracle Support and My Oracle Support Integration:** Detail the workflow for logging a Service Request (SR): gathering diagnostic artifacts (AWR/ADDMM reports, Alert Log excerpts, IPS packages), setting severity levels, and submitting tickets via MOS. Include a sample SR description referencing lab artifacts and workflow screenshots.
- e) Manage and Apply Patches:** Run opatch lsinventory from the OS terminal to display patch levels and Release Updates (RU). Document the complete patch deployment process:
- Download patch archive from MOS.
 - Run prerequisite checks (opatch prereq).
 - Apply patch (opatch apply).
 - Execute post-installation scripts (datapatch -verbose).
- Include opatch lsinventory terminal output and documented patch steps.
- f) Validation and Cleanup:** Summarize environment status (patch level, open incidents, security registration). Clean up temporary tablespaces, test tables, and generated IPS packages. Include final status verification queries and cleanup screenshots.